

# BST

Querying a BST  $\begin{cases} \rightarrow \text{look for a certain key } k \\ \rightarrow \text{min, max, predecessor, or successor} \end{cases}$

Search : Return a pointer to a node with key  $k$  if one exists, else return NIL.

Tree-Search ( $x, k$ )

If  $x == \text{NIL}$  or  $k == x.\text{key}$   
return  $x$

If  $k < x.\text{key}$

return Tree-Search( $x.\text{left}, k$ )

else

return Tree-Search( $x.\text{right}, k$ )

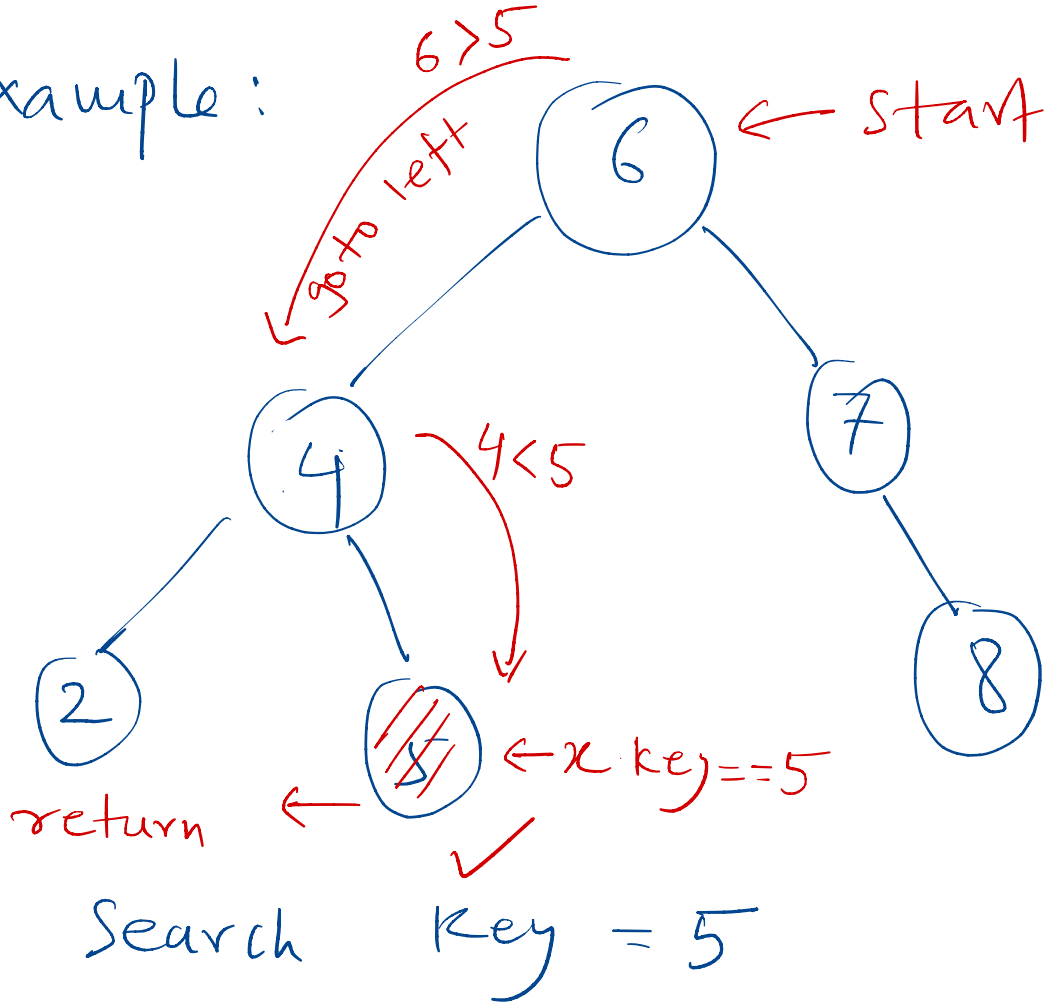
Call: Tree-Search( $T.\text{root}, k$ )

$\rightarrow \theta(h)$  complexity.

Example:

```
graph TD; 6((6)) --> 4((4)); 6 --> 7((7)); 4 --> 2((2)); 4 --> 5(((5))); 7 --> 8((8));
```

Search Key = 5



## Iterative -Tree -Search ( $\alpha, k$ )

while  $x \neq \text{NIL}$  and  $k \neq x.\text{key}$

if  $x \in x.\text{key}$

$$x = x \cdot \text{left}$$

else

$$x = x.\text{right}$$

return  $x$

Node \*n = T.root; → copy

Call: Iterative-Tree-Search ( $n, k$ );

int k = 1; → Pass by reference →

int &i = k; ? vs int \*i = &k;

func (int &r) {

→  
Pass by reference

=====  
}

int j = 2;  
func(j);

func (int r) {

→  
Pass by value

=====  
}

func(j)

## Minimum / Maximum

→ returns a pointer to min/max element in the subtree rooted at given node  $x$  (assumed to be non-null)

---

### Tree - Minimum ( $x$ )

---

while  $x \cdot \text{left} \neq \text{NIL}$

|  $x = x \cdot \text{left}$

return  $x$

---

---

### Tree - Maximum ( $x$ )

---

while  $x \cdot \text{right} \neq \text{NIL}$

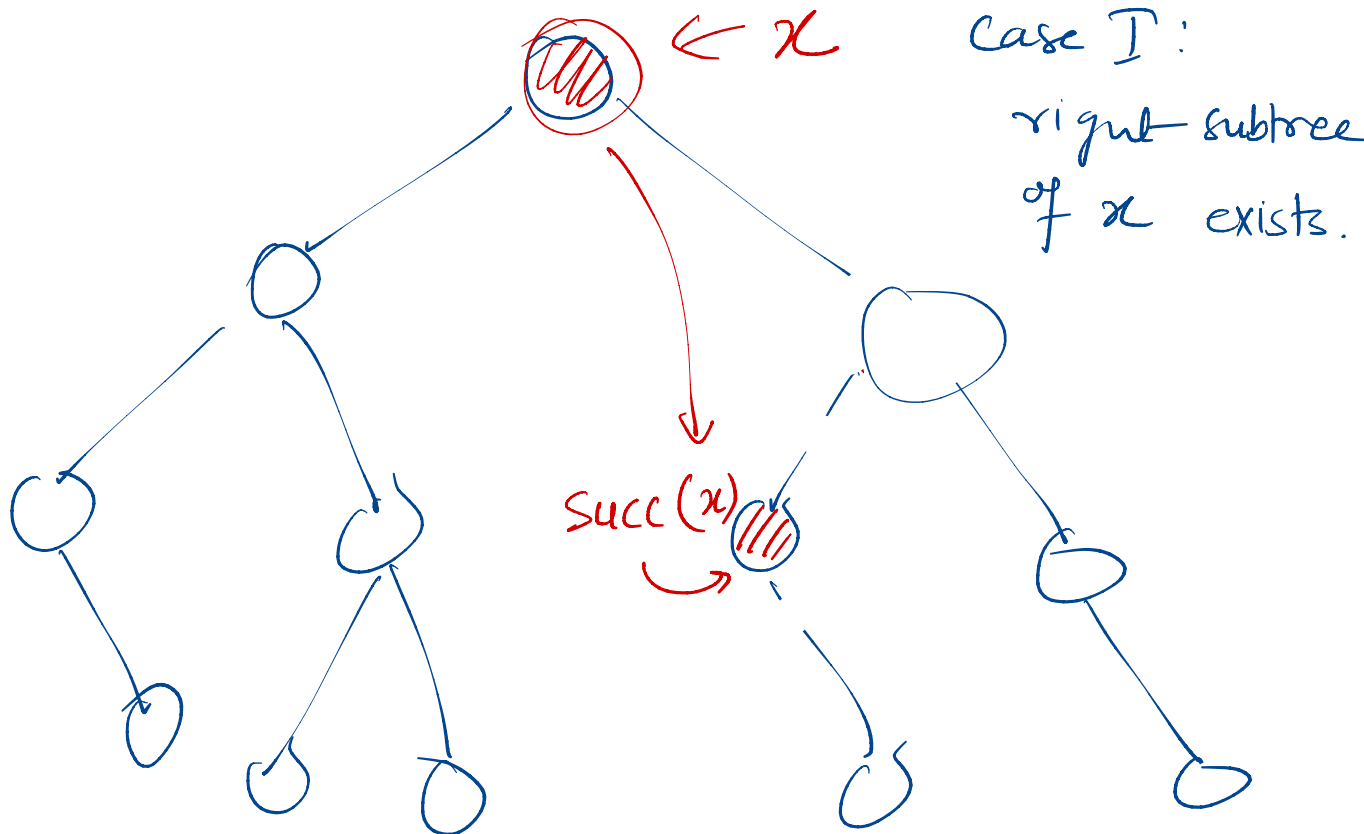
|  $x = x \cdot \text{right}$

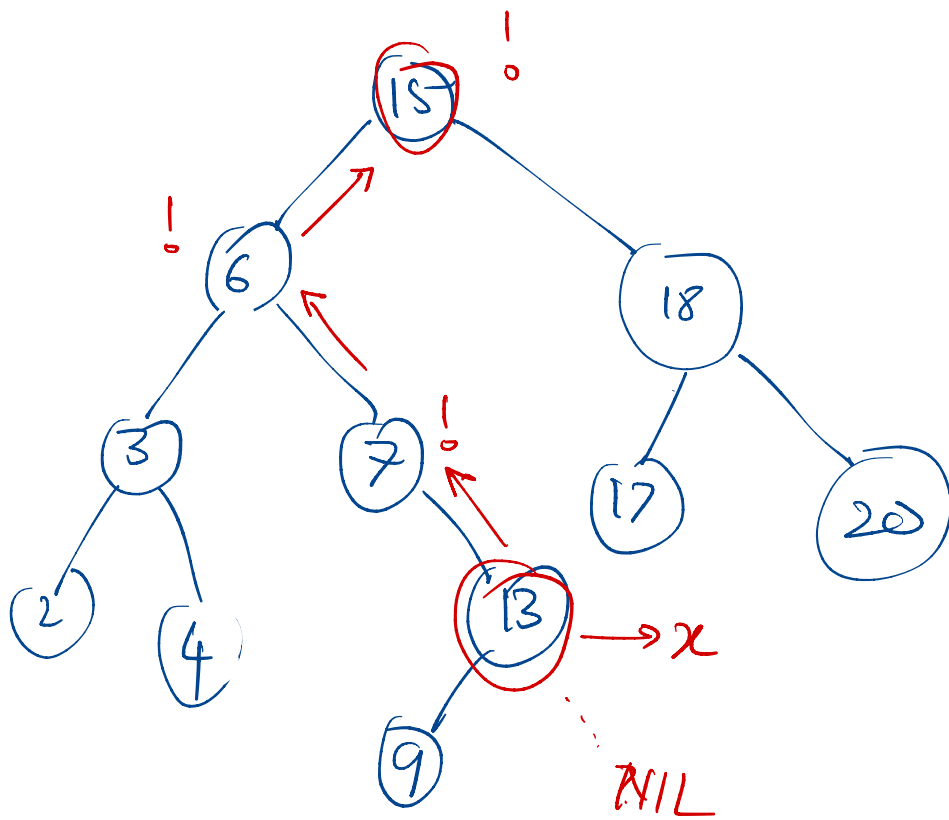
return  $x$

---

# Successor & Predecessor

- Finds successor of a given node in the sorted order determined by the inorder walk.
- Successor of node  $x$  = node with the smallest key larger than  $x.key$
- Define successor as the next node to be visited in an inorder tree walk





Case II : right  
subtree of  
 $x$  doesn't  
exist

lowest  
ancestor whose  
left child is  
also an ancestor  
of  $x$

## Tree - Successor ( $x$ )

if  $x.\text{right} \neq \text{NIL}$

return Tree - Minimum( $x.\text{right}$ )

else

$y = x.p$

while  $y \neq \text{NIL} \ \& \ x == y.\text{right}$

$x = y$

$y = y.p$

return  $y$